CMPSC 463: Design and Analysis of Algorithms

Project 2

Emergency Aid Distribution and Route Optimization System

Julia Skrentkovych, Tedi Shaho

Janghoon Yang

April 26, 2026

Table of Contents

I. PROJECT DESCRIPTION 3

II. PROJECT SIGNIFICANCE 3

III. CODE STRUCTURE 5

IV. DESCRIPTION OF ALGORITHMS 8

V. VERIFICATION OF ALGORITHMS 10

VI. RESULTS AND ANALYSIS 18

VII. AI USE 22

VIII. CONCLUSION 23

I. PROJECT DESCRIPTION

The objective of the project is to implement a web-based humanitarian response simulation that is designed for use by an emergency aid distribution center to allocate limited supplies to affected locations. It will determine optimal delivery routes to affected locations, maximizing efficiency with respect to distance and time. The system models locations as graph nodes and roads as weighted edges. It prioritizes locations based on urgency and demand, allows different optimization modes such as urgency-first, distance-first, and balanced, and applies shortest-path and greedy decision-making strategies to produce an optimized delivery plan under crisis constraints. The system has several features such as adding/managing affected locations, setting supply availability (food, medicine, etc.), and assigning urgency levels to locations. Then, it will compute priority scores and generate optimized delivery routes with proper visualization as a graph. More importantly, the simulation dynamically updates every time so that users receive the most up-to-date information on delivery routes, priority locations based on urgency, and total travel distance. The simulation is designed specifically for user interaction, and it allows for manual input on supplies, locations, urgency, roads, etc. In order to visualize how everything is connected, a network graph is used, where the edges connect one location to another based on routes. This project is inspired by real-world challenges in conflict or post-conflict environments, where infrastructure may be damaged, resources are limited, and decisions must be made quickly. The simulation uses various algorithms learned in the course, as well as other data structures. Lastly, various technologies were used in the project. HTML, CSS and JavaScript were used for the frontend, and Python for the backend.

II. PROJECT SIGNIFICANCE

The Emergency Aid Distribution and Route Optimization System is a project that can prove meaningful in many ways. A real-world scenario in which the system can be helpful is during a war or conflict, as

such environments require frequent emergency interventions for both people and infrastructure. People who reside near a zone where a war is unfolding are at risk of serious injury, therefore first-aid resources must be delivered to them at an optimal time. In addition, food and water can be scarce in such zones, so such a system can be very useful in identifying the zones at risk and delivering the resources. More importantly, the system will ensure that each time supplies are to be allocated, the most efficient route will be utilized for delivery in order to save as much time as possible. Various factors play a role in determining route efficiency, such as supply availability, urgency levels, road distance, and condition. Unfortunately, during times of conflict, there can be innocent people whose lives are at risk, therefore a primary goal of such an application is to always reduce the chances of death or injury while maximizing distribution of limited supplies to those in need as quickly as possible.

In addition to its practical applications, the system introduces a unique combination of features, including dynamic scenario input, real-time updates, and multiple optimization modes that allow users to prioritize urgency, distance, or a balance of both. The integration of graph-based visualization with algorithmic decision-making provides a more interactive and adaptable approach compared to traditional methods.

III. CODE STRUCTURE

Systematic Code Structure Diagram

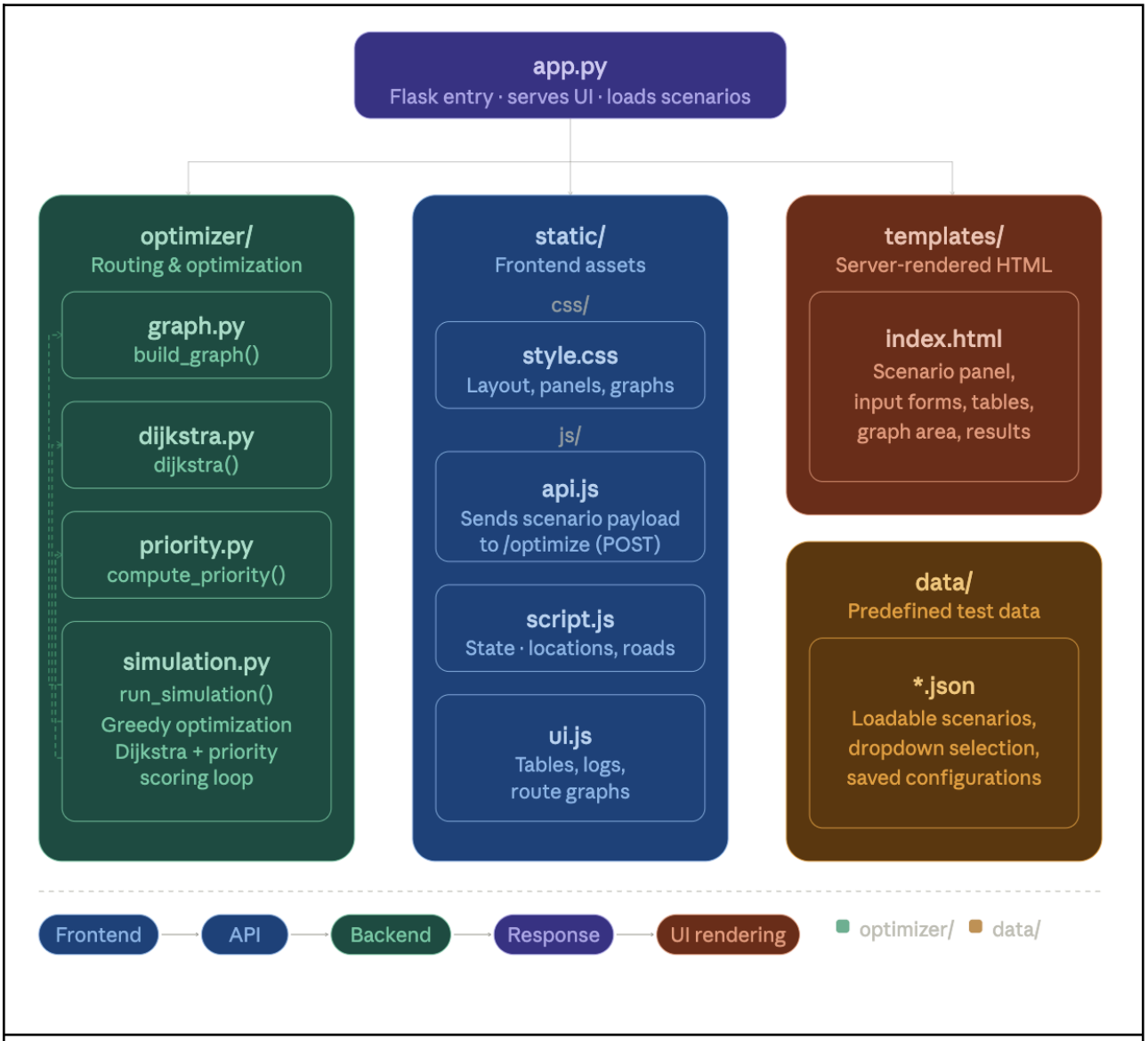

Figure a - System Architecture Showing Frontend, API Layer, and Backend with Integrated Optimization Modules

Code Structure Explanation

1. **Backend (Flask Application)**

The backend is handled by **app.py**, which serves as the main entry point of the system. It is responsible for rendering the user interface (UI), loading predefined scenarios, and handling requests from the frontend. When the user runs the optimization, the frontend sends a request to the **/optimize** endpoint, and **app.py** passes this data to the optimization logic. After the computation is complete, the results are returned to the frontend in **JSON** format.

2. **Optimization Module (optimizer/)**

The **optimizer/** folder contains the main algorithm logic used in the system. Each file has a specific role, and they work together to perform the optimization.

- **graph.py** builds a weighted graph from the road data. Each location is treated as a node, and each road is represented as an edge. Blocked roads are ignored, and damaged roads are assigned a higher cost to reflect slower or more difficult travel.
- **dijkstra.py** implements Dijkstra's algorithm to find the shortest path between two locations. This is used repeatedly during the optimization to determine the most efficient route between points.
- **priority.py** computes a priority score for each location based on its urgency level and the amount of required supplies. This allows the system to favor more critical locations.
- **simulation.py** is the main optimization engine. It combines the graph, shortest path calculations, and priority scores into a greedy decision-making loop. At each step, the system selects the next location based on the chosen optimization mode, calculates the shortest path to it, and distributes available supplies. It also tracks unmet demand and detects unreachable locations. The final output includes the delivery order, total travel distance, and supply distribution results.

3. **Frontend (static/ and templates/)**

The frontend handles user input and displays the results of the optimization.

- **index.html** defines the structure of the UI, including the scenario panel, input forms, tables, graph area, and results section.
- **style.css** controls the visual design of the page, including layout, panels, tables, badges, and graph styling.
- **api.js** sends the scenario data to the backend using a **POST** request to the **/optimize** endpoint and receives the computed results.
- **script.js** manages the main state of the application, including locations, roads, and supplies. It also handles user actions such as adding or deleting data, resetting the system, and loading scenarios.
- **ui.js** is responsible for updating the interface after the optimization runs. It renders the results table, summary information, simulation logs, and both the all routes graph and the optimized route graph.

4. **Data Directory (data/)**

The **data/** folder contains predefined scenario files in JSON format. These scenarios allow users to quickly test the system with different configurations of locations, supply demands, and road conditions. They can be loaded directly through the dropdown menu in the interface.

5. **System Flow**

The system follows a clear and structured flow. The user enters or loads a scenario on the frontend, and the data is sent to the backend through an API request. The backend processes the data using a greedy optimization approach that combines Dijkstra's shortest path algorithm with priority scoring. The computed results are then returned to the frontend and displayed through

tables, summary cards, logs, and graph visualizations. This separation between data input, processing, and visualization makes the system easier to understand and maintain.

IV. DESCRIPTION OF ALGORITHMS

For this project, various algorithms learned during the course were implemented in order to effectively integrate all of the features of the simulation. Since the simulation includes multiple scenarios with different constraints (worst infrastructure, worst supply shortage, etc), most algorithms were chosen to provide efficient and practical solutions under the given constraints.

Graph Representation

Due to the nature of this simulation, visualization of the network of routes and locations is fundamental in order to ensure the correct and optimal resource distribution. A graph is a suitable data structure to represent all of this information. For this simulation specifically, an undirected weighted graph is implemented using an adjacency list. An empty graph is initialized, which will store the nodes and their respective neighbors. Then, for each road in the graph, the distance between connected locations is stored as edge weights. In addition, the simulation also considers the status of a road, meaning whether it is open or blocked, as well as whether it is damaged or not. If a road is open, then it is included, otherwise it is ignored by the graph, since it cannot be used for resource allocation. A realistic feature of the graph representation is that it adjusts the travel cost of a road if it is damaged. If there is no damage, then the cost remains the same as the distance, and there is no change. However, if there is a road with damaged conditions, then it becomes 50% more expensive, and that is shown in the graph. Lastly, since it is an undirected graph, edges are added in both directions, so if road X connects to Y, then Y connects to X, as well.

Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path from a source node to all other nodes in a weighted graph with non-negative edge weights. It works by keeping track of candidate nodes and always selecting the one with the smallest known distance at each step. While this choice is locally optimal each time, the algorithm still guarantees a globally optimal shortest path. This is different from the greedy delivery strategy used later, which makes decisions step by step without guaranteeing a globally optimal result. In this simulation, Dijkstra's algorithm is used multiple times. It is first used from the distribution center to find the initial distances, and then used again between locations as the delivery process moves from one stop to the next. All nodes start with a distance of infinity, except the source, which starts at zero. As the algorithm explores neighbors, it updates distances whenever a shorter path is found. The process continues until all reachable nodes have been finalized. One important detail is that if a direct path between two locations has the same cost as a longer path with multiple steps, the system chooses the direct path. This helps avoid unnecessary routing through other locations when a simpler route is just as efficient.

Priority Queue

In cases where resources are limited, prioritization is required to ensure that locations with the greatest need are served first. Therefore, a priority queue is used in this system. The 'heapq' module was used for implementation. A priority queue always returns the smallest, or highest-priority, element first based on the defined comparison.

In this project, the priority queue is used to order locations based on priority score and optimization mode. The queue compares elements (stored as tuples) element by element, allowing the system to prioritize locations according to urgency, demand, or distance depending on the selected mode. This enables efficient decision-making when selecting the next location to serve.

Additionally, in Dijkstra's algorithm, the priority queue is used to select the node with the shortest distance so far. This ensures that the closest nodes are explored first and guarantees that the shortest paths are computed efficiently.

Greedy Optimization Strategy

In addition to shortest path computation, the system uses a greedy strategy to determine the order in which locations are served. At each step, the algorithm selects the next location based on the chosen optimization criteria.

Once a location is selected, the system computes the shortest path to it using Dijkstra's algorithm and allocates available resources. This process is repeated until all locations are visited or all resources are used up. Although this approach does not guarantee a globally optimal solution in all cases, it provides an efficient and practical method for handling changing constraints and limited resources.

V. VERIFICATION OF ALGORITHMS

Toy Verification Scenario

The following toy scenario is used to verify correctness of the system's algorithms. The scenario includes one distribution center and three locations: Shelter 1, Clinic 1, and Village 1, connected by roads with simple distances.

Graph Representation Verification

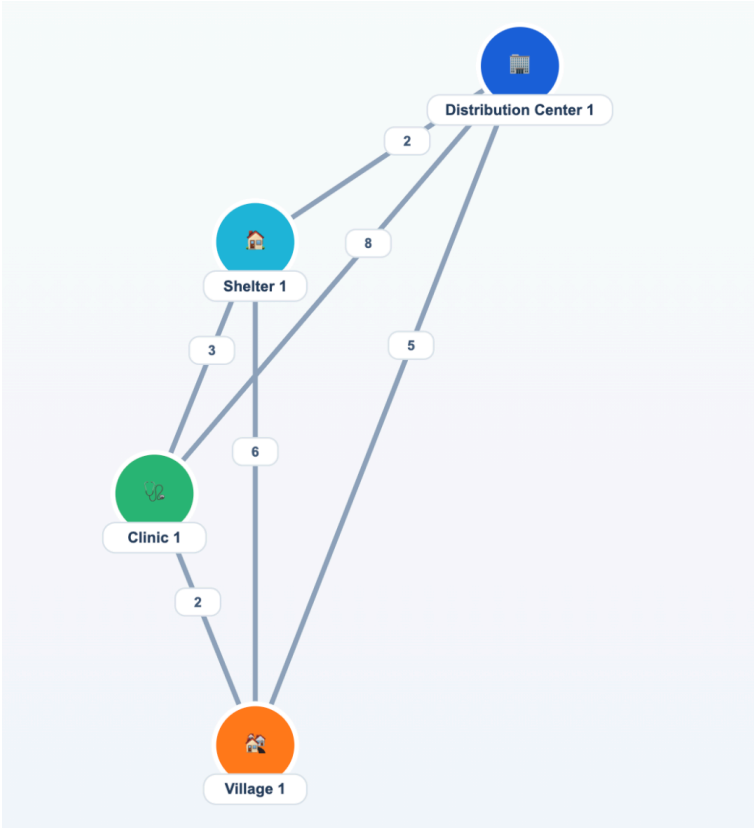

Figure b - Graph Representation of Locations and the Paths Between Them

The graph representation is verified using the toy scenario above. Each location is correctly represented as a node, including the Distribution Center (D.C. 1), Shelter 1, Clinic 1, and Village 1. The roads between locations are represented as edges, with each edge assigned a weight corresponding to its distance.

Since all roads in this scenario are open, each connection is included in the graph, and edges are added in both directions to represent an undirected graph. This ensures that movement between locations is possible in both directions, which is consistent with the undirected graph model. The graph structure

shown above matches the defined scenario, confirming that nodes, edges, and weights are correctly constructed.

Dijkstra Verification

Dijkstra's algorithm is verified by checking whether the system selects the shortest path between two locations. In the toy graph, Clinic 1 can be reached from D.C. 1 in two ways.

The direct path: Distribution Center 1 → Clinic 1 (total distance: 8).

The indirect path: Distribution Center 1 → Shelter 1 → Clinic 1 (total distance: 2 + 3 = 5).

Since 5 is less than 8, the shortest path should be through Shelter 1. As shown in the optimized graph, the algorithm selects the path through Shelter 1 instead of the direct route. This confirms that Dijkstra's algorithm is working correctly, as it evaluates multiple paths and chooses the one with the minimum total cost.

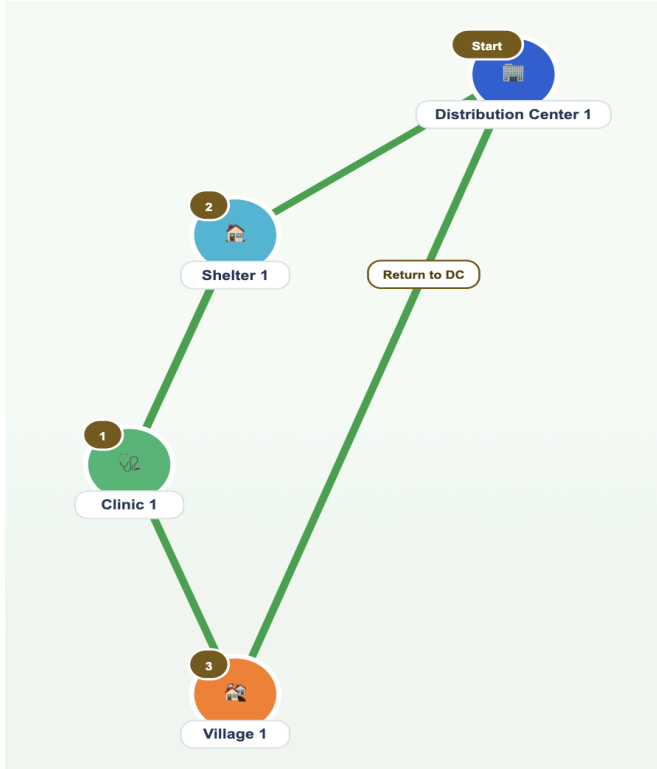

Figure c - Optimized Graph Showing Shortest Path Selection from Distribution Center 1 to Clinic 1 Using Dijkstra's Algorithm

<u>Additional Note:</u>

For this verification, the focus is only on shortest path calculation, not on the order in which locations are visited. The node ordering shown in the graph is part of the greedy optimization strategy and is not relevant in this verification. Additionally, the 'Return to DC' label is not considered in this verification either, as it is related to route completion rather than shortest path computation.

The connection between Clinic 1 and Village 1 appears in the optimization graph because the system computes shortest paths between locations during route construction. While Dijkstra's algorithm is initially used to determine shortest paths from the D.C. 1, it is also applied between intermediate

locations as part of the greedy optimization process. Therefore, this edge is not part of the initial shortest path from the distribution center, but is still a valid shortest path used during routing.

Priority Score Verification

The priority scoring is verified by checking whether the system correctly assigns higher scores to locations with higher urgency and demand. The priority score is calculated using urgency weights combined with total resource demand, which allows the system to rank locations based on need.

Locations & Needs

Name	Type	Food	Medicine	Water	Blankets	Urgency	Action
Shelter 1	Shelter	80	20	60	25	High	Delete
Clinic 1	Clinic	30	100	20	10	Critical	Delete
Village 1	Village	120	25	90	35	Low	Delete

Figure d - Locations and Demand Data Used for Priority Score Calculation

In the toy scenario, Clinic 1 has the highest urgency level (Critical) and a high demand for medicine, while Shelter 1 has High urgency, and Village 1 has Low urgency but a larger overall demand.

Results Table

Rank	Location	Type	Priority	Route	Distance	Food	Medicine	Water	Blankets	Supply Plan Status
1	Clinic 1	Clinic	280	Distribution Center 1 → Shelter 1 → Clinic 1	5	30 / 30	100 / 100	20 / 20	10 / 10	Planned Full Supply
2	Shelter 1	Shelter	260	Clinic 1 → Shelter 1	3	80 / 80	20 / 20	60 / 60	25 / 25	Planned Full Supply
3	Village 1	Village	297	Shelter 1 → Clinic 1 → Village 1	5	120 / 120	25 / 25	90 / 90	35 / 35	Planned Full Supply

Figure e - Results Table Showing Computed Priority Scores and Final Location Ranking

As shown in the results table above, the priority scores reflect both urgency and demand, while the final ranking reflects the selected optimization mode.

Additional Note:

Even though Village 1 has a higher numerical priority score, it is ranked last because the system is using urgency mode. In this mode, urgency level is prioritized over the numerical score, meaning that locations with higher urgency are visited first regardless of differences in computed priority values.

This confirms that the priority scoring and urgency-based prioritization are working correctly, as the system correctly identifies and prioritizes the most critical locations.

Greedy Verification

The greedy optimization strategy is verified by comparing the visit order under different optimization modes. The same toy scenario is used, but the system is run twice: once in urgency-first mode and once in distance-first mode.

In urgency-first mode, the system prioritizes locations with higher urgency levels. In the toy scenario, Clinic 1 has Critical urgency, Shelter 1 has High urgency, and Village 1 has Low urgency. Therefore, Clinic 1 is selected first, even though it is not the closest location.

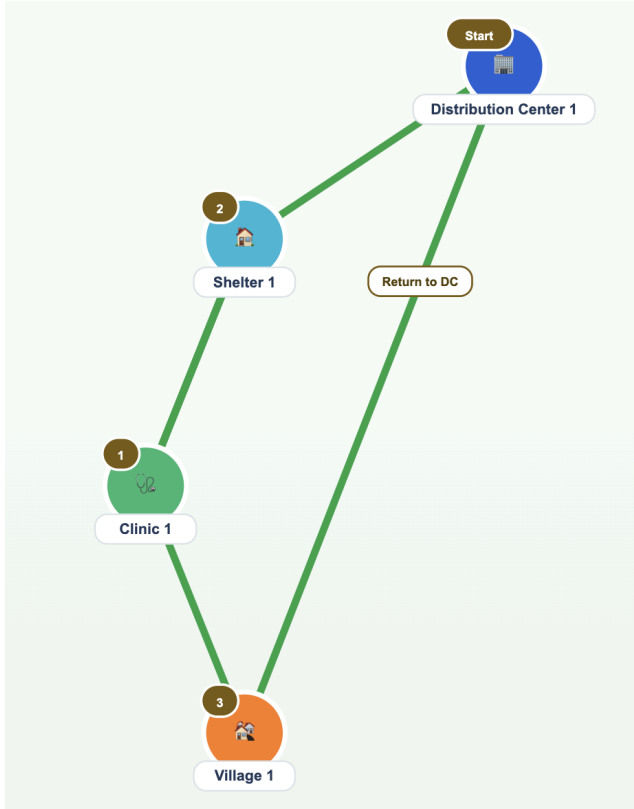

Figure f - Optimized Graph Showing Shortest Path Selection in Urgency-First Mode

Delivery Order: Distribution Center 1 → Clinic 1 → Shelter 1 → Village 1 → Distribution Center 1

Actual Travel Path: Distribution Center 1 → Shelter 1 → Clinic 1 → Shelter 1 → Clinic 1 → Village 1 → Distribution Center 1

Note: some locations may appear in the travel path before their delivery number because the route may pass through them to reach another location first.

Figure g - Delivery Order and Actual Travel Path Generated by the Greedy Optimization Strategy in Urgency-First Mode

In distance-first mode, the system prioritizes the closest reachable location first. Since Shelter 1 is closest to D.C. 1 with the distance of 2, it is selected before the other locations.

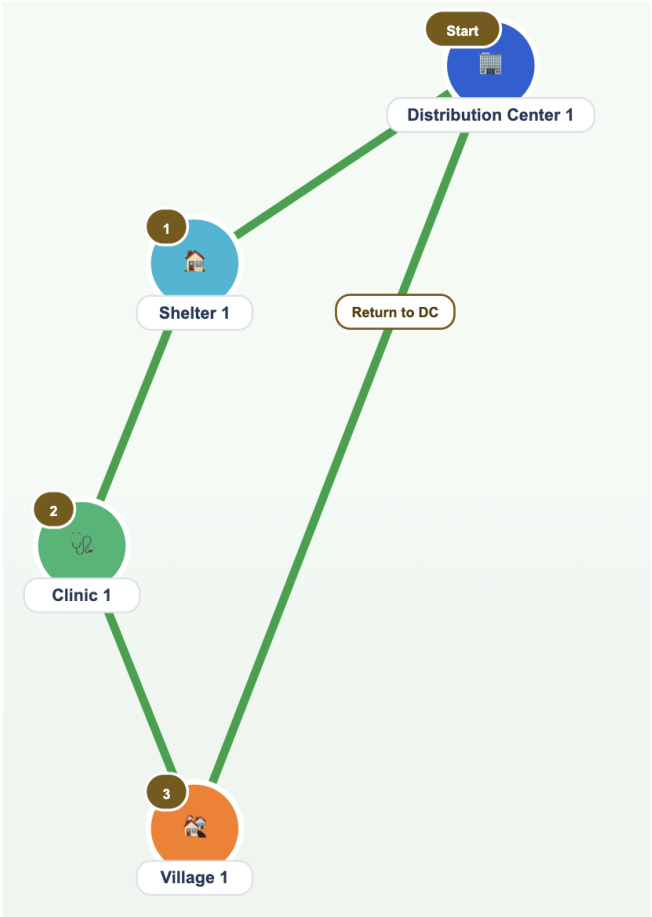

Figure h - Optimized Graph Showing Shortest Path Selection in Distance-First Mode

Delivery Order: Distribution Center 1 → Shelter 1 → Clinic 1 → Village 1 → Distribution Center 1
Actual Travel Path: Distribution Center 1 → Shelter 1 → Clinic 1 → Village 1 → Distribution Center 1
Note: some locations may appear in the travel path before their delivery number because the route may pass through them to reach another location first.

Figure i - Delivery Order and Actual Travel Path Generated by the Greedy Optimization Strategy in Distance-First Mode

This comparison verifies that the greedy strategy is working correctly because the order of visited locations changes based on the selected optimization mode. The algorithm does not use one fixed order

for every case. Instead, it adapts its decision-making based on the optimization criteria selected by the user.

VI. RESULTS AND ANALYSIS

To analyze how the algorithm behaves under constraints, the system is tested using the Scarce Network scenario with the optimization mode set to distance-first. This means locations are prioritized based on proximity instead of urgency. The scenario includes six locations, including two shelters, two clinics, and two villages. All of them are connected to the Supply Center, with additional connections between shelters and clinics, and between clinics and villages. The available supplies are intentionally limited compared to the total demand, so running out of resources becomes an important factor in the results.

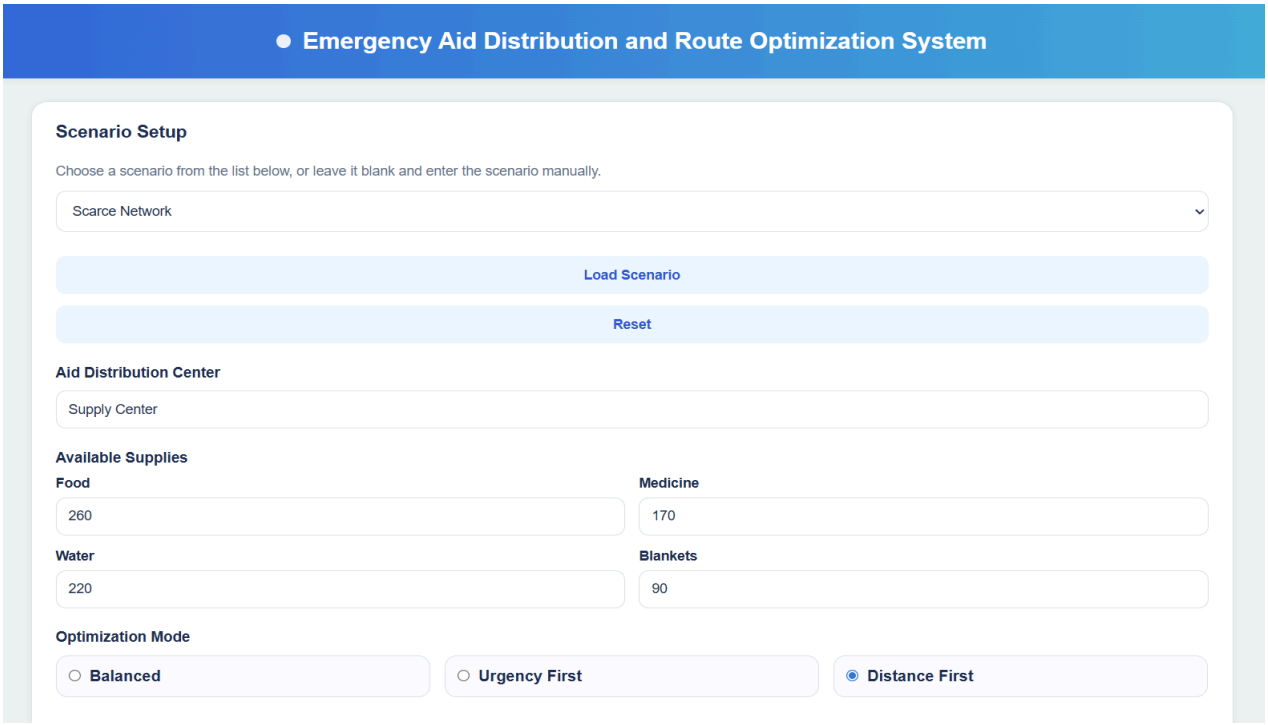

Figure j - Scenario Setup and Input Configuration for Distance-First Optimization Mode

The full network graph is generated first, showing all possible routes between the Supply Center and the six locations. Each node represents a location, and each edge represents a road, with weights showing

the travel distance. All ten roads in this scenario are open, so no connections are removed during graph construction. This confirms that the system correctly represents the network structure before any optimization is applied.

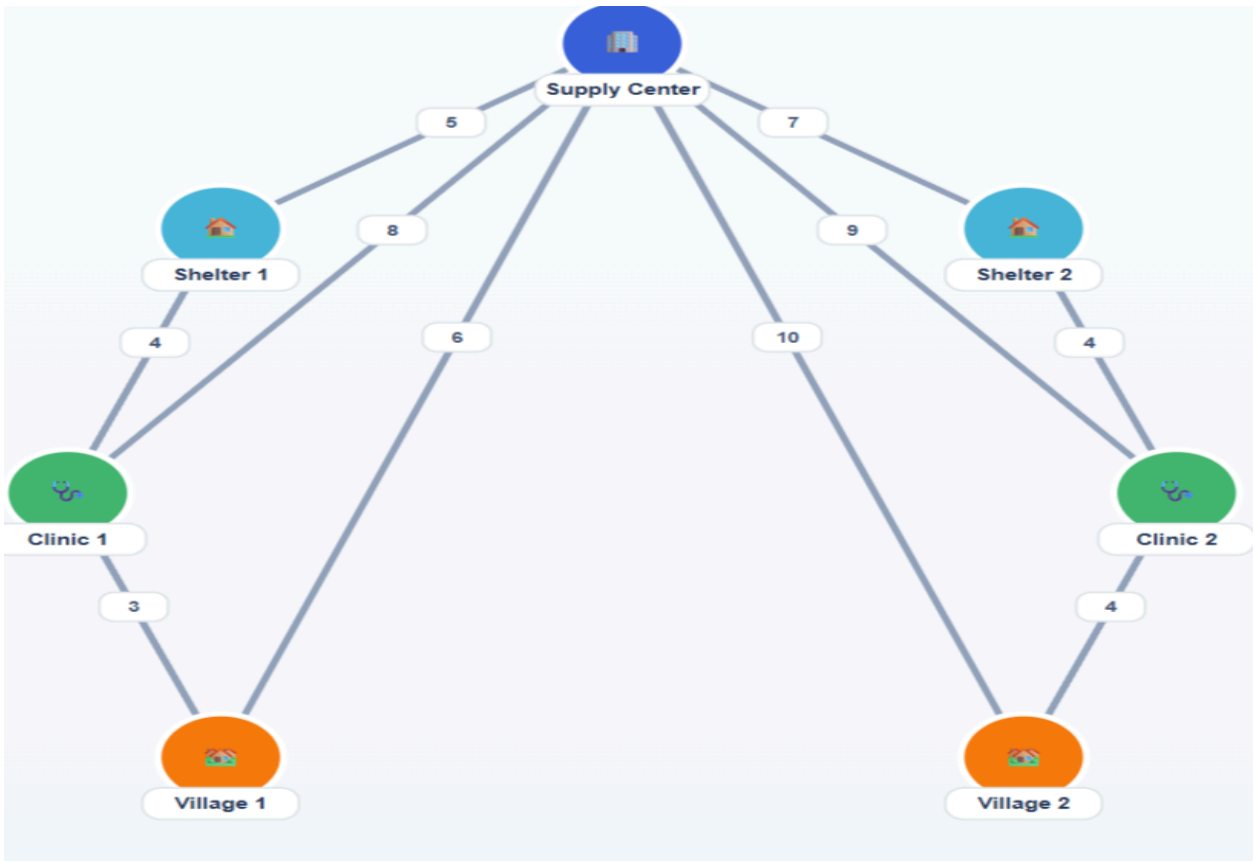

Figure k - Full Network Graph of Locations and Paths Between Them

The optimization is then executed. In this scenario, every location is directly reachable from the Supply Center, so Dijkstra's algorithm determines the initial distances using just one edge. The distances from the Supply Center are: Shelter 1 (5), Village 1 (6), Shelter 2 (7), Clinic 1 (8), Clinic 2 (9), and Village 2 (10). Since the system is using distance-first mode, it always selects the closest reachable location at each step. However, each decision is made from the current location, not from the Supply Center. After

visiting Shelter 1, the next decision is made starting from Shelter 1. From there, Clinic 1 is only 4 units away, making it the closest option at that point. Because of this, the final order is not just based on sorting distances from the starting point. Instead, it follows a step-by-step nearest-location choice. The resulting delivery order is: Supply Center → Shelter 1 → Clinic 1 → Village 1 → Shelter 2 → Clinic 2 → Village 2 → Supply Center.

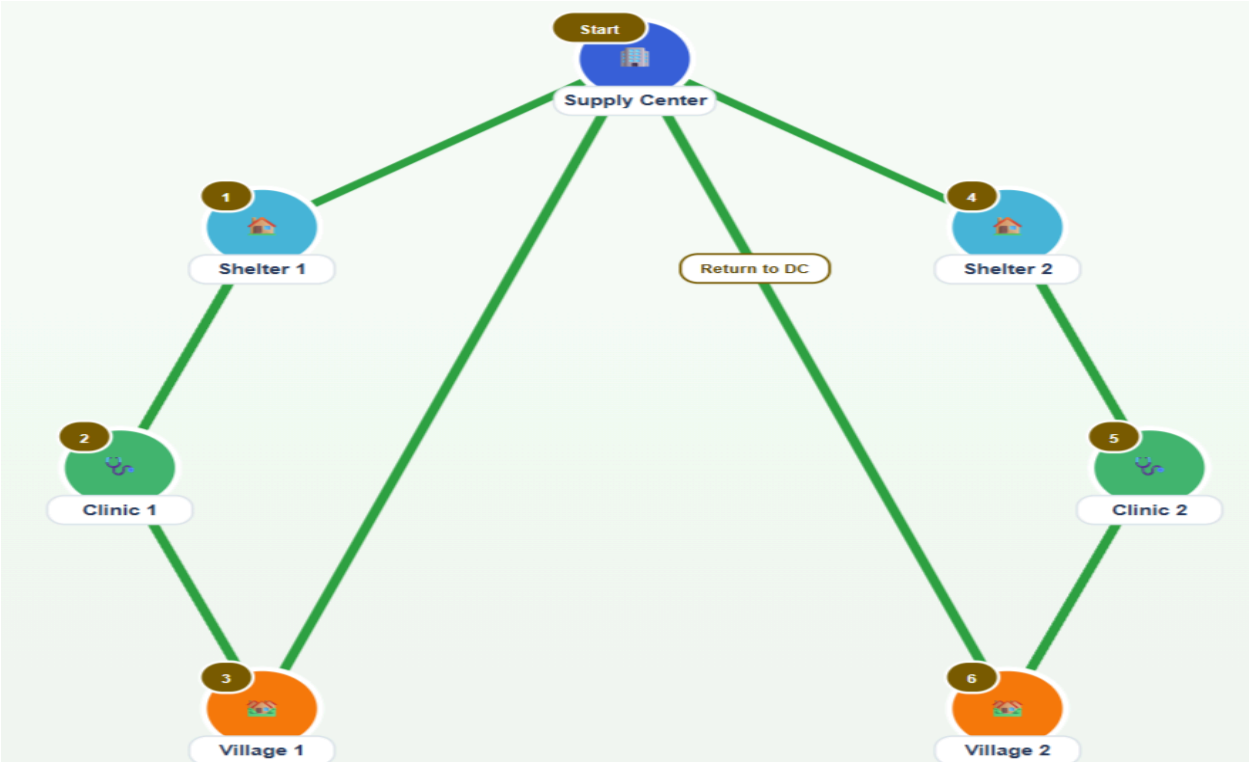

Figure l - Optimal Route Graph Showing Final Delivery Order and Visit Sequence in Distance-First Mode

The results table shows how the limited supplies are distributed across the delivery order. The total available supplies at the start are 260 food, 170 medicine, 220 water, and 90 blankets. The first three locations, Shelter 1, Clinic 1, and Village 1, are fully served and use up most of the available resources. By the time the system reaches rank 4, Shelter 2 only receives a partial allocation. Clinic 2 and Village

2, which are ranked 5th and 6th, receive no supplies. The status column clearly reflects these outcomes, showing ‘Planned Full Supply,’ ‘Planned Partial Supply,’ and ‘Reachable, No Supplies Left.’ This helps distinguish between locations that could not be reached and those that were reachable but could not be served due to limited resources.

Results Table

Rank	Location	Type	Priority	Route	Distance	Food	Medicine	Water	Blankets	Supply Plan Status
1	Shelter 1	Shelter	311	Supply Center → Shelter 1	5	110 / 110	25 / 25	70 / 70	30 / 30	Planned Full Supply
2	Clinic 1	Clinic	274	Shelter 1 → Clinic 1	4	30 / 30	95 / 95	20 / 20	10 / 10	Planned Full Supply
3	Village 1	Village	284	Clinic 1 → Village 1	3	100 / 100	30 / 30	80 / 80	25 / 25	Planned Full Supply
4	Shelter 2	Shelter	240	Village 1 → Supply Center → Shelter 2	13	20 / 90	20 / 20	50 / 60	25 / 25	Planned Partial Supply
5	Clinic 2	Clinic	209	Shelter 2 → Clinic 2	4	0 / 25	0 / 70	0 / 20	0 / 0	Reachable, No Supplies Left
6	Village 2	Village	216.5	Clinic 2 → Village 2	4	0 / 85	0 / 20	0 / 65	0 / 0	Reachable, No Supplies Left

Figure m - Results Table Showing Computed Priority Scores and Final Location Ranking in Distance-First Mode

This outcome highlights an important trade-off in distance-first optimization. Clinic 2 has High urgency but still receives no supplies because it is ranked fifth based on distance. If urgency-first mode were used instead, the system would prioritize Clinic 1 and Clinic 2 earlier due to their Critical and High urgency levels. This could lead to better outcomes for the most at-risk locations, but it would likely increase the total travel distance. Because of this, distance-first mode works best when minimizing travel is the main goal, while urgency-first mode is more suitable when addressing the most critical needs is more important than keeping the route short.

The delivery order and the actual travel path are shown below. The actual path may go through some locations more than once, since Dijkstra’s algorithm finds the shortest path between each pair of stops

separately. Because of this, a location that was already visited can still appear again along the route to the next destination, even though it is not served again.

Delivery Order: Supply Center → Shelter 1 → Clinic 1 → Village 1 → Shelter 2 → Clinic 2 → Village 2 → Supply Center
Actual Travel Path: Supply Center → Shelter 1 → Clinic 1 → Village 1 → Supply Center → Shelter 2 → Clinic 2 → Village 2 → Supply Center
Note: some locations may appear in the travel path before their delivery number because the route may pass through them to reach another location first.

Figure n - Delivery Order and Actual Travel Path Generated by the Optimization Algorithm in Distance-First Mode

Overall, the results show that the system correctly combines shortest path computation with a greedy allocation approach under the selected optimization mode. The partially served and unserved locations are expected due to limited supplies, not because of any issue with the algorithm. This reflects real situations where difficult decisions have to be made about how to distribute resources when there is not enough to meet all demands.

VII. AI USE

AI tools were used throughout the development of this project as a supporting resource, mainly to assist with frontend design, debugging, and code refinement.

At the beginning, AI was used to generate an initial frontend layout, including the structure of the HTML interface and organization of input forms and panels. This served as a starting point and was later manually modified to fit the project's requirements and achieve the desired design.

AI was also used during development to help with debugging and logic checks. It assisted in identifying potential issues and suggested fixes, especially when working with interactions between the frontend and backend components.

In addition, AI tools were used for final code cleanups, such as removing redundant or no longer used code and improving readability and organization.

VIII. CONCLUSION

This project shows how algorithmic techniques can be applied to a real-world humanitarian aid distribution problem under resource and infrastructure constraints. The system is able to generate optimized delivery plans by combining graph-based modeling, shortest path computation, priority scoring, and a greedy allocation approach. The three optimization modes – urgency-first, distance-first, and balanced – produce different outcomes, showing that the choice of strategy directly affects which locations receive aid and in what order.

Several challenges were encountered during development. Handling blocked and damaged roads required two different approaches during graph construction, since blocked roads must be removed completely, while damaged roads only require an increase in travel cost. It was also important to clearly separate cases where locations were unreachable from cases where supplies were simply exhausted. In addition, maintaining consistent data flow between the frontend, API, and backend required debugging, especially when working with JSON field names.

This project provided a practical application of key concepts from the course. Graph representation using adjacency lists was used to model the network and its connections. Dijkstra's algorithm was used multiple times within the system, showing how it can be applied as part of a larger process. The priority queue supported both shortest path computation and decision-making in the optimization process. Finally, the greedy strategy demonstrated how choosing locally optimal decisions at each step can lead to efficient overall behavior, while still involving trade-offs.